CO1 Activity 2: Information Theory & Training/Validation Curves

```python
import numpy as np

# True distribution P and predicted distribution Q
P = np.array([0.5, 0.3, 0.2])
Q = np.array([0.4, 0.4, 0.2])

# Calculate Entropy H(P) in bits (base 2)
_entropy = -np.sum(P * np.log2(P))

# Calculate Cross-Entropy H(P, Q) in bits (base 2)
_cross_entropy = -np.sum(P * np.log2(Q))

# Calculate KL Divergence D_KL(P || Q) in bits (base 2)
_kl_divergence = np.sum(P * np.log2(P / Q))

print(f"True Distribution P: {P}")
print(f"Predicted Distribution Q: {Q}\n")
print(f"Entropy H(P): {_entropy:.4f} bits")
print(f"Cross-Entropy H(P, Q): {_cross_entropy:.4f} bits")
print(f"KL Divergence D_KL(P || Q): {_kl_divergence:.4f} bits")

# Verify relationship: H(P, Q) = H(P) + D_KL(P || Q)
assert np.isclose(_cross_entropy, _entropy + _kl_divergence)
print("\nVerification: H(P, Q) = H(P) + D_KL(P || Q) holds true!")
```

```
True Distribution P: [0.5 0.3 0.2]
Predicted Distribution Q: [0.4 0.4 0.2]

Entropy H(P): 1.4855 bits
Cross-Entropy H(P, Q): 1.5219 bits
KL Divergence D_KL(P || Q): 0.0365 bits

Verification: H(P, Q) = H(P) + D_KL(P || Q) holds true!
```

```python
import matplotlib.pyplot as plt

# Let's simulate loss trends across epochs for three models with varying capacities
epochs = np.arange(1, 41)

# 1. Underfitting (Capacity too low: both training and validation errors remain high)
train_loss_under = 0.5 * np.exp(-0.02 * epochs) + 0.3
val_loss_under = 0.52 * np.exp(-0.02 * epochs) + 0.31

# 2. Optimal Fit (Optimal capacity: both losses decrease and stabilize close to each other)
train_loss_opt = 0.7 * np.exp(-0.1 * epochs) + 0.05
val_loss_opt = 0.72 * np.exp(-0.1 * epochs) + 0.08

# 3. Overfitting (Capacity too high: training loss goes to zero, validation loss bounces back up)
train_loss_over = 0.8 * np.exp(-0.15 * epochs) + 0.01
val_loss_over = 0.82 * np.exp(-0.15 * epochs) + 0.03 + 0.0008 * (epochs ** 2)

# Plotting the curves
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot Underfitting
axes[0].plot(epochs, train_loss_under, 'r-', label='Train Loss')
axes[0].plot(epochs, val_loss_under, 'b--', label='Val Loss')
axes[0].set_title("Underfitting (Low Capacity)")
axes[0].set_xlabel("Epochs")
axes[0].set_ylabel("Loss")
axes[0].grid(True)
axes[0].legend()

# Plot Optimal Fit
axes[1].plot(epochs, train_loss_opt, 'g-', label='Train Loss')
axes[1].plot(epochs, val_loss_opt, 'b--', label='Val Loss')
axes[1].set_title("Optimal Fit (Good Capacity)")
axes[1].set_xlabel("Epochs")
axes[1].set_ylabel("Loss")
axes[1].grid(True)
axes[1].legend()

# Plot Overfitting
axes[2].plot(epochs, train_loss_over, 'g-', label='Train Loss')
axes[2].plot(epochs, val_loss_over, 'r--', label='Val Loss')
axes[2].set_title("Overfitting (Excessive Capacity)")
axes[2].set_xlabel("Epochs")
axes[2].set_ylabel("Loss")
axes[2].grid(True)
axes[2].legend()

plt.tight_layout()
plt.show()
```

